

Final Project Proposal - kpublic1

Background

Web-based programs such as the Expasy Translate Tool offered by the SIB Swiss Institute of Bioinformatics (<https://web.expasy.org/translate/>) are fast and useful tools for genetic analysis pipelines, especially when it comes to novel DNA sequences that will be used for protein prediction and construction. Multiple sites like these are available on the Internet, but I want to synthesize my own knowledge by building one from scratch for the final project. The aim is to provide simple, efficient DNA or RNA sequence translation on the class server with minimal resources used.

Functionality

This tool will have one main input field, where a user will be able to input a DNA (A, T, C, G) or RNA (A, U, C, G) sequence. When the user clicks "Translate", it will process each codon (3-base segment) to convert the nucleotide sequence into an amino acid sequence with one-letter codes. This output will be displayed in a box below the input, and a key will be provided on the right side of the page to show the one-letter amino acid codes and their corresponding amino acid names for user benefit. To fulfill the database component, up to 5 of the user's previous entries will be stored in a MySQL database and accessed by the user.

Description

The file structure of the tool will be as follows:

- /var/www/html/sequence-translator/
 - index.cgi (handles server-side logic)
 - templates/
 - main.html (main page template)
 - error.html (error page template)
 - static/
 - css/
 - style.css (page styling)
 - js/
 - translator.js (client-side Javascript/jQuery)
 - sql/
 - schema.sql

index.cgi

- This is a Python CGI script, and it will handle all server-side functionality. Its main function is to handle translation requests. Upon receiving one from POST data, it sanitizes and validates the input, detects whether it's DNA (ATCG) or RNA (AUCG), and performs codon-by-codon translation using the standard codon table, which will be stored as a dictionary. It will then save the input and output to the SQL database, with up to 5 entries being kept; if there are to be more than 5 entries, the oldest will be deleted. It then returns a JSON response containing the protein sequence and updated history. It

also performs history requests, either retrieving the information from a previous entry or clearing all history from the database.

templates/main.html

- This is the primary HTML5 template file. The head includes links to the CSS style sheet and script tags for jQuery and the custom JavaScript file. The body contains the main input area, including a textarea for the sequence entry and a “Translate” button. Below this is the output box where the protein sequence result is displayed. As described, the right side of the page will contain a reference key for one-letter amino acid codes and their corresponding full names. The template will also contain a history section that displays up to five of the user’s previous translations.

		Second letter					
		U	C	A	G		
First letter	U	UUU } Phe UUC } UUA } Leu UUG }	UCU } UCC } Ser UCA } UCG }	UAU } Tyr UAC } UAA Stop UAG Stop	UGU } Cys UGC } UGA Stop UGG Trp	U C A G	Third letter
	C	CUU } CUC } Leu CUA } CUG }	CCU } CCC } Pro CCA } CCG }	CAU } His CAC } CAA } Gln CAG }	CGU } CGC } Arg CGA } CGG }	U C A G	
	A	AUU } AUC } Ile AUA } AUG Met	ACU } ACC } Thr ACA } ACG }	AAU } Asn AAC } AAA } Lys AAG }	AGU } Ser AGC } AGA } Arg AGG }	U C A G	
	G	GUU } GUC } Val GUA } GUG }	GCU } GCC } Ala GCA } GCG }	GAU } Asp GAC } GAA } Glu GAG }	GGU } GGC } Gly GGA } GGG }	U C A G	

- Above is the standard codon table that will be used for this translation tool. Source link: <https://openstax.org/books/biology/pages/15-1-the-genetic-code>

templates/error.html

- This is the template for a simple page that will display in case of any errors, including a user-friendly error message. It will open when encountering validation errors, database connection failures, or other issues.

static/css/style.css

- This stylesheet will define the visuals and layout, including the main content area, the input and output sections, and the amino acid key. It also styles the textarea, buttons, output display box, and the history section.

static/js/translator.js

- This JavaScript file will use jQuery to handle the client-side interactions. When the user clicks “Translate”, it captures the input sequence, performs validation by checking for empty input and invalid characters, and sends a POST request to the CGI script with the

sequence data. When the response returns, it parses the JSON and updates the output box with the translated protein sequence, along with refreshing the history section to show the newest entry. In addition, it handles selections of history entries, sending requests to retrieve past translations and populating the input and output fields with said data. It also provides error handling.

sql/schema.sql

- This file will contain the MySQL schema for the user history database. It includes commands to create the database and a table for the translation history, including input sequences, sequence types, and output protein sequences. The planned format is as follows:

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each translation record
input_seq	VARCHAR(64)	NOT NULL	The original DNA or RNA sequence entered by the user
seq_type	ENUM('DNA', 'RNA')	NOT NULL	Indicates whether the input was DNA (containing T) or RNA (containing U)
seq_length	INT	NOT NULL	Number of nucleotides in the input sequence (useful for quick display without recounting)
output_protein	TEXT	NOT NULL	The resulting amino acid sequence in one-letter codes